

```
(%o3)          {- 4, 0, 6, 7, 32}
(%i4) S: adjoin(3, S); /* Updating S */
(%o4)          {- 4, 0, 3, 6, 7, 32}
(%i5) adjoin(7, S);
(%o5)          {- 4, 0, 3, 6, 7, 32}
(%i6) disjoint(7, S);
(%o6)          {- 4, 0, 3, 6, 32}
(%i7) disjoint(5, S);
(%o7)          {- 4, 0, 3, 6, 7, 32}
(%i8) quit();

$ maxima -q
(%i1) S: {-4, 0, 3, 6, 7, 32};
(%o1)          {- 4, 0, 3, 6, 7, 32}
(%i2) S1: subset(S, evenp);
(%o2)          {- 4, 0, 6, 32}
(%i3) powerset(S1);
(%o3)          [{}, {- 4}, {- 4, 0}, {- 4, 0, 6}, {- 4, 0, 6, 32}, {- 4, 0, 32}, {- 4, 6}, {- 4, 6, 32}, {- 4, 32}, {0}, {0, 6}, {0, 6, 32}, {0, 32}, {6}, {6, 32}, {32}]
(%i4) S2: {-35, -26, 0, 7, 32, 100};
(%o4)          {- 35, - 26, 0, 7, 32, 100}
(%i5) symmdifference(S1, S2);
(%o5)          {- 35, - 26, - 4, 6, 7, 100}
(%i6) symmdifference(S, S2);
(%o6)          {- 35, - 26, - 4, 3, 6, 100}
(%i7) quit();
```

Advanced set operations

With Maxima, much more than this can be done with sets, using just the advanced functionalities provided by it. So now, let's take a journey through them.

Cartesian product: Given 'n' sets, the function *cartesian_product()* returns a set of lists formed by the Cartesian product of the 'n' sets. The following demonstration explains what this means:

```
$ maxima -q
(%i1) cartesian_product({0, 1, 2}, {a, b, c});
(%o1)          {[0, a], [0, b], [0, c], [1, a], [1, b], [1, c], [2, a], [2, b], [2, c]}
(%i2) cartesian_product({0, 1}, {a, b}, {X, Y});
(%o2)          {[0, a, X], [0, a, Y], [0, b, X], [0, b, Y], [1, a, X], [1, a, Y], [1, b, X], [1, b, Y]}
(%i3) cartesian_product({0, 1}, {a, b, c});
(%o3)          {[0, a], [0, b], [0, c], [1, a], [1, b], [1, c]}
(%i4) quit();
```

Set partitions: Given a set S, it can be partitioned into various subsets, based on various mathematical principles. Maxima provides a host of functions for such partitioning—the basic one being *set_partitions()*. It returns a set of all possible partitions of the given set. With a number as the second argument, it gives only the partitions with that exact

number of sets in each partition. Shown below are some examples to make sense of this concept:

```
$ maxima -q
(%i1) S: {a, b, c};
(%o1)          {a, b, c}
(%i2) set_partitions(S);
(%o2)          {{{a}, {b}, {c}}, {{a}, {b, c}}, {{a, b}, {c}}, {{a, b, c}}, {{a, c}, {b}}, {{b, c}, {a}}, {{a, b, c}}}
(%i3) set_partitions(S, 1);
(%o3)          {{{a, b, c}}}
(%i4) set_partitions(S, 2);
(%o4)          {{{a}, {b, c}}, {{a, b}, {c}}, {{a, c}, {b}}}
(%i5) set_partitions(S, 3);
(%o5)          {{{a}, {b}, {c}}}
(%i6) set_partitions(S, 4);
(%o6)          {}
(%i7) belln(3);
(%o7)          5
(%i8) cardinality(set_partitions(S)); /* Number of elements */
(%o8)          5
(%i9) belln(4);
(%o9)          15
(%i10) belln(5);
(%o10)         52
(%i11) belln(6);
(%o11)         203
(%i12) quit();
```

In the above examples, *belln()* or the nth Bell number is the number of partitions of a set with 'n' members.

integer_partitions(n) is a specific function, which partitions a given positive integer 'n' into a set of positive integers, the sum of which adds up to the original integer. *num_partitions(n)* returns the number of such partitions returned by *integer_partitions(n)*. Examples follow:

```
$ maxima -q
(%i1) integer_partitions(1);
(%o1)          {[1]}
(%i2) num_partitions(1);
(%o2)          1
(%i3) integer_partitions(2);
(%o3)          {[1, 1], [2]}
(%i4) num_partitions(2);
(%o4)          2
(%i5) integer_partitions(3);
(%o5)          {[1, 1, 1], [2, 1], [3]}
(%i6) num_partitions(3);
(%o6)          3
(%i7) integer_partitions(4);
(%o7)          {[1, 1, 1, 1], [2, 1, 1], [2, 2], [3, 1], [4]}
(%i8) num_partitions(4);
```